Menos *macroblocking* en video en vivo sobre SRT: I-headers entregados por el edge

Vicente Andrade Ercilla
Universidad Técnica Federico Santa María, Chile
IPD438 Seminario de Redes de Computadores

Resumen

En video en vivo sobre UDP (*User Datagram Protocol*), un paquete perdido produce *macroblocking*: bloques corruptos que se propagan hasta el próximo cuadro intra, hasta 3 s después con un GoP (*Group of Pictures*) de 90 cuadros. SRT (*Secure Reliable Transport*) retransmite, pero solo dentro de su latencia; lo que no llega a tiempo se descarta. Proponemos que el edge, el servidor que reparte el video, reutilice los I-headers de AGiLE, copias intra de algunos P que ya viajan en el stream, y los entregue solo al espectador que descartó paquetes. No se pide nada al codificador ni se agrega latencia. El edge detecta el descarte con la marca de tiempo de los ACK (*acknowledgement*) de SRT. En un emulador, con la latencia por defecto de SRT, el I-header reduce a la mitad la duración del *macroblocking* en los cortes que SRT no recupera (de 1,1 a 0,5 s por evento). El costo es un 68 % más de tasa entre el codificador y el edge, una deriva leve después de recuperar y un peor resultado con cortes cortos, que SRT ya cubre.

Palabras clave: H.264, SRT, pérdida de paquetes, *macroblocking*, edge, GoP.

I. Introducción

En video en vivo, el codificador intercala cuadros intra (I), que se decodifican solos, y cuadros predichos (P), que solo describen lo que cambió respecto del anterior. Un GoP largo ahorra bits, pero si se pierde un paquete, el error se copia de P en P hasta el próximo intra y el espectador ve *macroblocking*. Sobre TCP (*Transmission Control Protocol*) la pérdida se convierte en pausas; este trabajo trata el transporte sobre UDP, que se usa cuando importa la latencia.

SRT [1] retransmite lo perdido, pero entrega cada paquete con una latencia fija L. Si un corte de la red dura más que L, el paquete se descarta y el *macroblocking* vuelve. Las salidas conocidas le cobran a todos los espectadores: subir L retrasa a todos, acortar el GoP sube la tasa de todos y pedirle un intra al codificador con un PLI (*Picture Loss Indication*) se lo envía a todos.

AGiLE [2] envía al edge, cada cierto número de P, un I-header: el mismo P codificado además como intra, dentro de una SEI (*Supplemental Enhancement Information*). Lo usa para que quien entra al canal no espere el próximo intra. Para el resto, ese I-header ya está en el edge y se descarta.

Objetivo. Que el espectador que perdió paquetes salga del *macroblocking* en el próximo I-header, sin pedir nada al codificador, sin agregar latencia y sin cambiar lo que recibe el espectador sin pérdidas. Las contribuciones son tres:

1. El edge le entrega el I-header solo al espectador que descartó paquetes (Sección V).
2. Una regla para que el emisor SRT sepa qué descartó el receptor, solo con la marca de tiempo de los ACK (*acknowledgement*) (Sección A).
3. Un emulador en C++ con un núcleo portable al STM32N6 y una comparación de UDP y SRT con y sin I-header, que incluye lo que cuesta el I-header (Sección VI).

La Sección II resume lo necesario de H.264 y SRT, y la Sección III los trabajos relacionados. La Sección IV plantea el problema, la Sección V la estrategia y la arquitectura, la Sección VI los resultados y la Sección VII las conclusiones.

II. Antecedentes

A. H.264

En H.264 [3], cada imagen se divide en macrobloques de 16×16 píxeles, agrupados en *slices*, y cada slice viaja en una NAL (*Network Abstraction Layer unit*). Un P copia cada macrobloque desde el cuadro anterior, desplazado según un vector de movimiento; si ese cuadro está dañado, la copia también. Un IDR (*Instantaneous Decoding Refresh*) es un intra que además vacía las referencias del decodificador. El I-header de AGiLE [2] es un intra no-IDR guardado en una SEI de datos de usuario, que el decodificador ignora si no la conoce. La SEI *recovery point* le indica al decodificador que puede retomar desde ese intra.

B. SRT en modo live

El video viaja en paquetes TS (*Transport Stream* de MPEG, *Moving Picture Experts Group*) de 188 B, siete por cada paquete SRT. El receptor de SRT [1] avisa cada hueco de inmediato con un NAK (*negative acknowledgement*) y lo repite periódicamente; el emisor retransmite con prioridad. Cada 10 ms, el receptor envía un ACK con el primer paquete que le falta. TSBPD (*Timestamp-Based Packet Delivery*) entrega cada paquete a la aplicación en su marca de tiempo más L, que por defecto es de 120 ms [1]. Si un paquete todavía falta en ese momento, TLPKTDROP (*Too-Late Packet Drop*) lo salta y

el ACK siguiente lo cubre como si hubiera llegado. Según el borrador, el emisor no se entera de ese salto [1].

III. Trabajos relacionados

AGiLE [2] usa un GoP de 10 s con un I-header cada 2 s, que el edge solo entrega a quien entra al canal. Frente a un GoP fijo de 2 s, logra un BD-rate (*Bjøntegaard Delta rate*, diferencia promedio de bits a igual calidad) de $-3{,}5$ %. Entrar por un I-header cuesta 1,1 puntos de VMAF (*Video Multimethod Assessment Fusion*), porque los P siguientes se calcularon sobre el P y no sobre el I-header. AGiLE no trata las pérdidas.

Keyframe injection [4, 5] mantiene un flujo aparte de intras, del que el reproductor pide uno al entrar o al perder paquetes, con una caída menor que 0,94 puntos de VMAF [4]. El estudio se extendió a tres generaciones de estándares de video [5], y una versión posterior repara con un P apoyado en un cuadro anterior a la pérdida, para evitar picos de tasa [6].

FrameTrace [7] mide en Media over QUIC la fracción de cuadros que se muestran. Con 1 % de pérdida informa VMAF 72,6 con GoP de 0,5 s y 55,7 con GoP de 2 s, y acorta el GoP cuando hay pérdida, lo que exige un lazo hasta el codificador, igual que el PLI de RTP (*Real-time Transport Protocol*).

En los trabajos revisados, la reparación sale de un flujo aparte o de un pedido al codificador. Aquí sale de un intra que ya está en el edge, y la decisión la toma el edge, sin un pedido del reproductor.

IV. Problema

Sean t_0 el instante en que el edge envía un paquete, d el retardo de ida y D la duración de un corte. El receptor solo puede pedir el paquete cuando el corte termina, y la retransmisión debe llegar antes del plazo $t_0 + d + L$. En primera aproximación, SRT recupera el corte si

$$D + \mathrm{RTT} + d \lesssim L, \tag{1}$$

donde RTT es el tiempo de ida y vuelta. Con $L = 120$ ms, $d = 15$ ms y RTT $= 30$ ms, SRT recupera cortes de hasta unos 75 ms. Haivision recomienda L entre 3 y 6 RTT para pérdidas de hasta 7 % [8], es decir, de 90 a 180 ms en este caso. Sea cual sea L, los cortes más largos que $L - \mathrm{RTT} - d$ se descartan: el slice llega truncado y el *macroblocking* dura hasta el próximo intra (Fig. 1).

Problema. Acortar el *macroblocking* de los cortes que SRT no recupera, en el tramo edge–espectador, con tres condiciones: no agregar latencia, no cambiar lo que recibe el espectador sin pérdidas y no pedir nada al codificador por cada pérdida. Suponemos que el tramo codificador–edge no pierde paquetes.

V. Estrategia y arquitectura

La Fig. 2 muestra la arquitectura. El codificador (un STM32N6570-DK, con codificador H.264 por hardware) envía al edge el stream con GoP largo y, cada n P, un I-header dentro de una SEI. El edge abre una conexión SRT con cada espectador. Al espectador sin pérdidas le quita las SEI, así que recibe exactamente el stream original. Cuando detecta que un

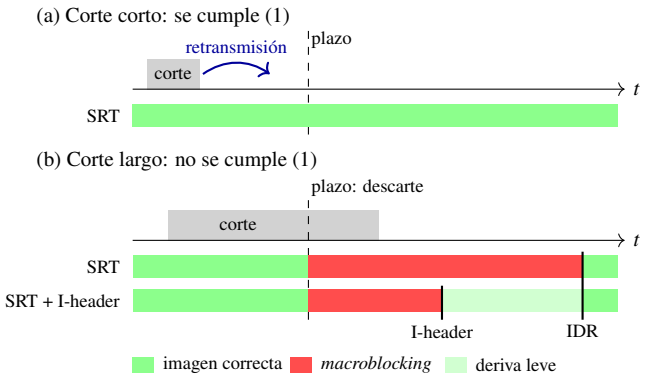


Fig. 1. Qué ve el espectador según la duración del corte. La barra gris es el corte de la red y la línea discontinua el plazo de entrega del primer paquete perdido. (a) La retransmisión llega antes del plazo y la imagen no se daña. (b) El paquete se descarta en su plazo. Con SRT solo, el *macroblocking* dura hasta el IDR; con I-header, hasta el primer I-header que el edge entrega después de detectar el descarte, seguido de una deriva leve hasta el IDR.

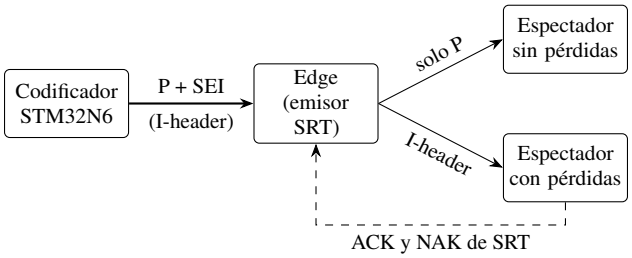


Fig. 2. Arquitectura. El I-header viaja siempre hasta el edge, que lo entrega solo al espectador que descartó paquetes. El edge lo sabe por los ACK y NAK de SRT, sin un mensaje nuevo.

espectador descartó paquetes, reemplaza el próximo P que tenga I-header por la SEI *recovery point* y el I-header, solo para ese espectador.

A. Cómo detecta el edge el descarte

El edge no recibe aviso del descarte, pero lo puede inferir. El receptor envía un ACK cada $T_{\mathrm{ACK}} = 10$ ms, y si salta un paquete, el ACK siguiente lo cubre igual. Ese ACK lleva la marca de tiempo de cuando se generó [1]. Un paquete s que el edge vio en un NAK se descartó si el primer ACK que lo cubre se generó después de su plazo y después de cuando se habría confirmado su última retransmisión:

$$g_s > \max\left(\tau_s,\ r_s + d + T_{\mathrm{ACK}}\right), \tag{2}$$

donde g_s es el instante de generación de ese ACK, $\tau_s = t_{0,s} + d + L$ el plazo de s y r_s el instante de su última retransmisión (si no hubo, solo cuenta τ_s). El segundo término evita marcar un paquete recuperado justo antes de su plazo. La regla supone que el edge no envía retransmisiones que ya no llegarían a tiempo.

B. Generación del I-header

En el STM32N6, la API (*Application Programming Interface*) del codificador permite pedir un intra no-IDR y fijar su QP (parámetro de cuantización). En el PC, libx264 no emite intras no-IDR sin cuadros B, así que el núcleo reescribe el encabezado de un IDR como intra no-IDR con el número de cuadro del P que reemplaza. El I-header debe tener el QP de ese P: con un

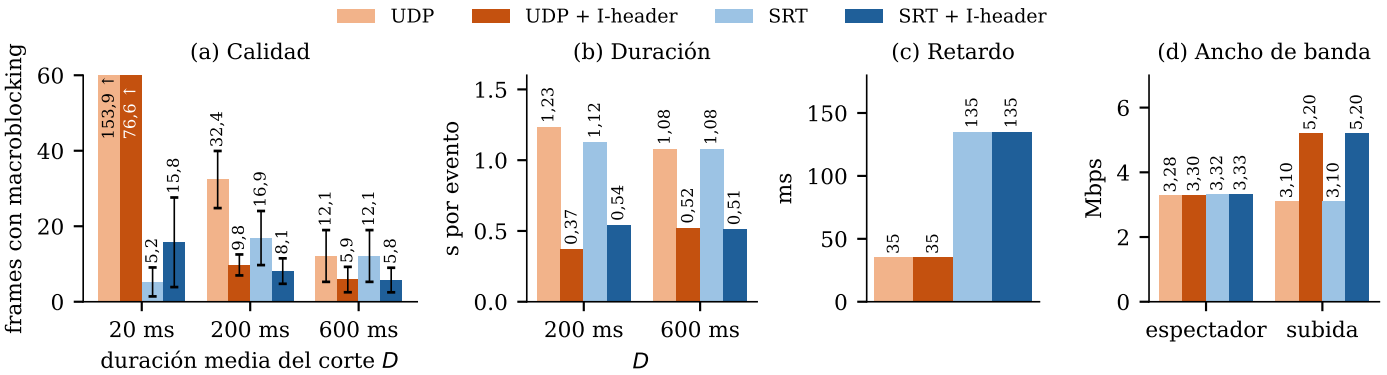


Fig. 3. Los cuatro casos con GoP 90 a 30 fps y $L = 120$ ms (media de 8 semillas de 10 s). (a) Cuadros con *macroblocking* ($\text{PSNR}_{\text{ref}} < 35$ dB) según la duración media del corte D, con su error estándar; las barras con flecha están recortadas. (b) Duración media del *macroblocking* por evento, en las semillas donde el caso sin I-header tuvo *macroblocking* (7 con UDP y 4 con SRT para $D = 200$ ms; 3 y 3 para $D = 600$ ms). (c) Retardo extremo a extremo fijado en el emulador. (d) Tasa hacia el espectador ($D = 200$ ms) y tasa de subida codificador–edge.

I-header de calidad fija, la deriva fue grande donde el codificador tenía pocos bits.

C. Emulador y métricas

El núcleo (C++17, sin memoria dinámica ni dependencias del sistema operativo) lee el H.264, arma la SEI, reescribe el encabezado y genera el TS; es la parte que correrá en el STM32N6, separada de la HAL (*Hardware Abstraction Layer*). En el PC, un simulador de eventos discretos modela SRT en modo *live* según el borrador [1]. El canal alterna entre un estado bueno y cortes sin entrega de duración media D (modelo de Gilbert–Elliott) y depende solo del tiempo y de una semilla, así que todos los casos ven los mismos cortes. Sin pérdida, el video decodificado es idéntico bit a bit al original.

La calidad de cada cuadro k se mide con el PSNR (*Peak Signal-to-Noise Ratio*) de la luminancia, a partir del error cuadrático medio (MSE, *Mean Squared Error*) entre la imagen de referencia Y_k y la recibida $\hat{Y}_k$, de $W \times H$ píxeles de 8 bits:

$$\text{PSNR}_k = 10 \log_{10} \frac{255^2}{\frac{1}{WH} \sum_{i,j} \left(Y_k(i,j) - \hat{Y}_k(i,j) \right)^2} \ [\text{dB}]. \qquad (3)$$

El "ruido" es la diferencia píxel a píxel. PSNR_{ref} compara con el mismo stream decodificado sin pérdida, así que solo mide lo que agregan la pérdida y la deriva. $\text{PSNR}_{\text{fuente}}$ compara con el video crudo. Un cuadro tiene *macroblocking* si $\text{PSNR}_{\text{ref}} <$ 35 dB, umbral elegido por nosotros. Un cuadro que no llega se reemplaza por el anterior. VMAF estima la calidad percibida (0 a 100) con un modelo entrenado con notas de observadores; todavía no lo medimos.

VI. Resultados

Configuración. Los primeros 300 cuadros de un clip, a 1280×720 y 30 fps, codificados con libx264 en perfil Baseline a 3 Mbps, con 45 slices por cuadro y GoP 90. Hay un I-header cada 5 P, con el QP del P que reemplaza. El retardo de ida es de 15 ms; SRT usa $L = 120$ ms y, sin SRT, el espectador tiene un búfer de 20 ms. Elegimos cortes a ambos lados de (1): de 20 ms (2 % del tiempo), que caben en L, y de 200 y 600 ms (4 %), que no caben. No afirmamos con qué frecuencia ocurren estos cortes en una red real. Con UDP + I-header, el espectador avisa cada descarte; con SRT + I-header, el edge usa la regla (2).

A. Calidad, latencia y ancho de banda

La Fig. 3 compara los cuatro casos. Sin retransmisión, el I-header reduce los cuadros con *macroblocking* a la mitad o menos con los tres tipos de corte. Con cortes de 20 ms, SRT solo deja 5,2 cuadros con *macroblocking*, y agregar el I-header lo empeora a 15,8: empeoró en 2 de las 8 semillas y mejoró en 1. Con cortes de 200 y 600 ms, SRT no recupera el corte en 4 y 3 de las 8 semillas. En esos eventos, el I-header baja la duración del *macroblocking* de 1,12 a 0,54 s y de 1,08 a 0,51 s. Con cortes de 600 ms, SRT no mejora a UDP.

El I-header no agrega retardo. La alternativa es subir L: con 250 ms desaparecen los eventos de 200 ms, pero los de 600 ms requieren 1000 ms, y ese retardo lo pagan todos los espectadores. La tasa hacia el espectador crece 0,3 % frente a SRT, porque el I-header solo baja cuando hay pérdida.

B. Qué perdemos

La Tabla I resume el costo del I-header. El mayor es la subida: un I-header cada 5 P lleva la tasa codificador–edge de 3,1 a 5,2 Mbps, aunque se paga una vez y no por espectador. Después de recuperar, el espectador no vuelve a la imagen exacta hasta el IDR. Para medir esa deriva sola, se forzó un I-header sin pérdida en 10 cuadros. Con QP cercano a 29, el PSNR_{ref} no bajó de 39,3 dB; con QP de 39 a 44, bajó a 28–35 dB. Contra la fuente, la diferencia fue de 1,0 a 1,4 dB.

C. Validez

Estos resultados tienen cuatro límites. Primero, son pocos eventos: 7 cortes no recuperados por SRT en 16 corridas de 10 s, con un solo clip. Segundo, el modelo de SRT no está validado contra libsrt y supone que el edge conoce sin error el instante de generación del ACK. Tercero, $L = 120$ ms queda bajo la recomendación de Haivision para 4 % de pérdida (180 ms) [8], aunque con 180 ms los cortes de 200 y 600 ms tampoco cumplen (1). Cuarto, el umbral de 35 dB cuenta como

Tabla I. Costo del I-header con GoP 90, frente al mismo caso sin I-header.

Costo	Valor
Tasa codificador–edge	+68 % (de 3,1 a 5,2 Mbps)
Tasa hacia el espectador	+0,3 % (SRT, $D = 200$ ms)
Retardo agregado	0
Deriva hasta el IDR	$\mathrm{PSNR}_{\mathrm{ref}} \geq 39{,}3$ dB con QP ≈ 29; 28–35 dB con QP 39–44; de $-1{,}0$ a $-1{,}4$ dB contra la fuente
Cortes cortos ($D = 20$ ms)	de 5,2 a 15,8 cuadros con *macroblocking*
Espera hasta recuperar	0,5 s por evento ($D = 200$ y 600 ms)
Pico de tasa al enviar el I-header	no medido
Cómputo en codificador y edge	no medido

macroblocking también la deriva fuerte del I-header, que no es un artefacto de bloques; separarlos requiere una métrica perceptual como VMAF.

VII. Conclusiones y trabajo futuro

SRT no elimina el *macroblocking*: quedan los cortes más largos que $L - \mathrm{RTT} - d$. En esos cortes, el I-header que ya está en el edge redujo a la mitad la duración del *macroblocking*, sin agregar latencia y con 0,3 % más de tasa hacia el espectador. El edge sabe a quién enviarlo sin cambiar SRT, por la marca de tiempo de los ACK. El costo está en la subida (+68 %) y en una deriva hasta el IDR. Con cortes cortos SRT basta y el I-header empeora el resultado, así que el edge debe enviarlo solo cuando el descarte es largo.

Como trabajo futuro quedan: (i) la misma comparación con GoP 30; (ii) más eventos, varios clips y la L recomendada; (iii) medir VMAF para separar deriva de *macroblocking*; (iv) una política del edge que no envíe el I-header con cortes cortos, con QP alto o con el IDR cerca; (v) validar el modelo contra libsrt sobre una red emulada; (vi) llevar el núcleo al STM32N6.

Referencias

[1] M. P. Sharabayko, M. A. Sharabayko, J. Dube, J. S. Kim y J. W. Kim, "The SRT protocol," IETF Internet-Draft draft-sharabayko-srt-01, trabajo en curso, 2021. [En línea]. Disponible: `https://datatracker.ietf.org/doc/draft-sharabayko-srt/`

[2] C. Chen, W. Yin, Z. Huang y S. Shi, "AGiLE: Enhancing adaptive GOP in live video streaming," en *Proc. ACM Multimedia Systems Conf. (MMSys '24)*, 2024, pp. 34–44, doi: 10.1145/3625468.3647609.

[3] ITU-T, "Advanced video coding for generic audiovisual services," Recomendación ITU-T H.264, ago. 2024. [En línea]. Disponible: `https://www.itu.int/rec/T-REC-H.264`

[4] G. Van Wallendael, H. Mareen, J. Vounckx y P. Lambert, "Keyframe insertion: Enabling low-latency random access and packet loss repair," *Electronics*, vol. 10, no. 6, art. 748, 2021, doi: 10.3390/electronics10060748.

[5] H. Mareen, M. Courteaux, P.-J. Speelmans, P. Lambert y G. Van Wallendael, "A study on keyframe injection in three generations of video coding standards for fast channel switching and packet-loss repair," *Multimedia Tools and Applications*, vol. 83, pp. 44485–44501, 2024, doi: 10.1007/s11042-023-17163-2.

[6] H. Mareen, P. Lambert y G. Van Wallendael, "P-frame injection for efficient packet-loss repair in ultra-low-latency video streaming," en *Proc. IEEE Int. Conf. Visual Communications and Image Processing (VCIP)*, 2023. [En línea]. Disponible: `https://ieeexplore.ieee.org/document/10402720/`

[7] B. Denizer, L. Winkel y O. Landsiedel, "FrameTrace: Frame-level telemetry for Media over QUIC," en *Proc. EMS '25, ACM SIGCOMM 2025 Workshops*, 2025, pp. 31–36, doi: 10.1145/3746441.3748231.

[8] Haivision, "SRT latency," en *SRT Protocol Documentation*, v1.5.3. [En línea]. Disponible: `https://doc.haivision.com/SRT/1.5.3/Haivision/latency`